Knowledge in a Nutshell: Automatyzacja & AI - dobre praktyki

Krótka ściąga wiedzy z teorii szkolenia dla BIM Meetup Polska.

1 4 Poziomy Automatyzacji, Kompas Decyzyjny i ROI

Zawsze wybieraj najprostsze narzędzie, które dowiezie oczekiwany efekt.

1. Prompt

`Gemini, Claude, ChatGPT`

Zadania jednorazowe (*ad-hoc*), wymagające ciągłego osądu człowieka.

2. Asystent AI

`Gems (Google), Custom GPTs`

Powtarzalne zadania o **stałej strukturze wyjścia** i zmiennym wsadzie. Konfigurujesz raz. **Reaktywny** (czeka na polecenie).

3. Workflow

`Power Automate, Apps Script, Make, n8n (+ LLM)`

Wieloetapowy proces łączący aplikacje o przewidywalnym przebiegu. LLM działa np. jako klasyfikator w jednym z kroków.

4. Agent AI

`Antigravity, Codex, Claude Code, Cursor`

Autonomiczny: dostaje **cel**, sam dekomponuje go na kroki, dobiera asystentów/narzędzia i reaguje na błędy w pętli zwrotnej.

*Język ma znaczenie: mówimy **agenty AI**, nie „agenci" (to narzędzia, jak piloty do TV vs piloci w kokpicie).*

Kompas decyzyjny: oceń proces w 5 pytaniach:

NR	PYTANIE DIAGNOSTYCZNE	1 PKT	2 PKT	3 PKT	4 PKT
P1	**Jak często to robisz?**	Ad-hoc	Miesięcznie	Tygodniowo	Codziennie
P2	**Czy struktura jest powtarzalna?**	Zawsze inna	Częściowo	Prawie identyczna	Zawsze identyczna
P3	**Ile czasu zajmuje jednorazowo?**	Minuty	Pół godziny	Godzina	Kilka godzin
P4	**Czy wymaga osądu W TRAKCIE?**	**Dużo osądu**	Trochę osądu	Minimalny osąd	**Mechaniczne**
P5	**Ile narzędzi/aplikacji łączysz?**	1 narzędzie	2 narzędzia	3 narzędzia	4+ narzędzi
Progi punktowe:		`5-8 pkt:` **Prompt**	`9-12 pkt:` **Asystent/Gem**	`13-16 pkt:` **Workflow**	`17-20 pkt:` **Agent AI**

(nawet przy 17–20 pkt sprawdź najpierw, czy nie wystarczy Workflow lub skrypt w Pythonie).

Uwaga do P4: Oceniasz potrzebę ludzkiego osądu **w trakcie trwania każdego kroku**, a nie tylko na starcie czy przy końcowej akceptacji.

Liczenie opłacalności i zwrotu z inwestycji - ROI:

Czas zaoszczędzony jednostkowo na zadaniu × **Powtarzalność (duża skala)** > **Koszt i czas: wdrożenia, weryfikacji i utrzymania.**

- **Mierz czas przed i po wdrożeniu.** Zanim wdrożysz system, automatyzację czy AI, zapisz, ile trwa zadanie dziś. Bez tego nie porównasz zysku i nie przełożysz go na godziny ani złotówki dla firmy.

2 BHP - Bezpieczeństwo Danych, AI Offline i Ograniczanie Ryzyka Błędów

Gdzie trafiają Twoje dane?

Darmowe czaty i licencje konsumenckie: Twoje dane trenują model.

Konta biznesowe, Enterprise i komercyjne API: Dane nie trenują modelu (przy Enterprise możliwe NDA), ale nadal **wychodzą do chmury**.

AI Offline (lokalnie - Ollama / LM Studio): Odłączasz Wi-Fi, a model pracuje na Twoim dysku. Zero wycieku, zero opłat za tokeny - idealne do poufnych umów, RODO i wstępnej obróbki plików.

Anonimizacja i deanonimizacja przed chmurą:

Gdy potrzebujesz mocy chmurowego modelu na wrażliwym tekście, skrypt w Pythonie (lub mały model lokalny) zamienia nazwy własne przed wysyłką, a po powrocie robi **deanonimizację**:

PRZED: `Jan Kowalski z POLBUD S.A. zgłosił roszczenie na 450 000 PLN dot. Port Gdynia`

PO: `[OSOBA_1] z [FIRMA_A] zgłosił roszczenie na [KWOTA_1] dot. [PROJEKT_X]`

Architektura procesu, CyberSecurity i AI Policy:

1. **Human-in-the-Loop (HITL) + bramki `STOP`:** Bot zawsze przygotowuje **DRAFT**, a człowiek klika **WYŚLIJ**. Przy niejasności agent ma zakaz zgadywania - **zatrzymuje proces (`STOP`) i pyta człowieka**.
2. **Agent-Audytor (Krytyk):** Drugi agent weryfikuje wynik pierwszego przed podjęciem akcji.
3. **Ochrona przed pętlami (Infinite Loops):** Zawsze taguj/oznaczaj przetworzone maile oraz pliki i ustaw **twarde limity kwotowe na koncie API**.
4. **Klucze API i Prompt Injection:** Sekrety trzymaj wyłącznie w zmiennych środowiskowych lub modułach sekretów (nigdy w kodzie ani plikach konfiguracji!). Uważaj na **Prompt Injection** ukryte w cudzych mailach lub plikach PDF.
5. **Pokora techniczna i EU AI Act:** Prototyp wyklikany z AI wymaga audytu IT/CyberSec przed produkcją. Zamiast zakazu AI (który rodzi *Shadow AI*), stwórz firmowe **AI Policy** (dozwolone narzędzia, zakazane dane, szkolenie, odpowiedzialność człowieka).

© 2026 Katarzyna Barcz AutoMATE | All Rights Reserved

3 Useful Links & Narzędzia na Start

- **Starter środowiska multiagentowego:** https://github.com/katarzynabarcz/bimmeetup-wro-202611
  Code → Download ZIP, otwórz folder w Antigravity, Claude Code lub Codexie i wklej zdanie z `START-PROMPT.md`.
- **Interaktywna Matryca Decyzyjna (Canvas App):** https://share.gemini.google/WPttrnjfA5Xx
  narzędzie do oceny procesów z zespołem (na podstawie programu *Google AI: Umiejętności Jutra 3.0*).
- **StackEdit:** https://stackedit.io/app#
  darmowy edytor Markdown w przeglądarce (surowy kod obok podglądu na żywo).
- **Markdown Guide:** https://www.markdownguide.org/
  szybka ściąga składni i przykładów.
- **Obsidian:** https://obsidian.md/
  lokalne foldery `.md`, graficzny podgląd YAML (*Properties*), linki `[[ ]]`, baza *Second Brain*.
- **Agent Skills:** https://agentskills.io
  otwarta specyfikacja folderu `SKILL.md` (*Progressive Disclosure*).

4 Struktura środowiska multiagentowego (OKF + starter)

Koniec z „wklejaniem w okienko czatu". Kontekst, zasady i procedury leżą w plikach na Twoim dysku, dzięki czemu zyskujesz **Zero Vendor Lock-in** (zmieniasz silnik AI, a środowisko zostaje). Jedna treść w `AGENTS.md` i `.agents/` działa w **Antigravity, Claude Code i Codexie** (Claude Code: `CLAUDE.md` z jedną linią `@AGENTS.md`).

```
my-project/
├── AGENTS.md          Orkiestrator: cel projektu, mapa folderów, zakazy globalne, router
├── .agents/           Konfiguracja wspólna dla wszystkich narzędzi
│   ├── rules/         Reguły: standardy i ograniczenia (Markdown + YAML)
│   ├── skills/        Procedury krok po kroku (SKILL.md + opcjonalnie scripts/)
│   └── agents/        Subagenty z osobnym oknem kontekstowym
├── setup/             Cele konfiguracji, dobre praktyki per narzędzie, nieaktywne wzory
├── knowledge/         Baza wiedzy OKF: fakty, słowniki, BEP/EIR (.md + YAML)
│   └── changelog/     Dziennik sesji: co zrobione, co dalej, decyzje
├── templates/         Wzory raportów i notatek (powtarzalny format wyjścia)
└── project/           Pliki robocze: wejścia, drafty, wyniki
```

5 Elementy układanki & Ściągawka decyzyjna

ELEMENT	GDZIE	KIEDY SIĘ ŁADUJE I PO CO?
Orkiestrator	`AGENTS.md`	**Czytany zawsze, na starcie każdej rozmowy.** Cel projektu, mapa folderów, zakazy globalne i router.
Reguła „zawsze"	`.agents/rules/` + wiersz w routerze	**Zawsze.** Dłuższe zasady obowiązujące w każdej rozmowie (np. wersjonowanie `_vXX.YY`).
Reguła „na zadanie / plik"	`.agents/rules/` + wiersz w routerze	**Gdy pasuje do zadania** lub typu pliku (np. styl konkretnego dokumentu).
Skill	`.agents/skills/nazwa/SKILL.md`	**Na żądanie.** Procedura krok po kroku (*Discovery → Activation → Execution*), opcjonalnie ze skryptami. Bez wiersza w routerze.
Subagent	`.agents/agents/`	**Na zlecenie Orkiestratora.** Osobny kontekst, model domyślnie dziedziczony (`model: inherit`), można dać lżejszy; oddaje tylko wynik (nie rozmawiasz z nim wprost).
Wiedza	`knowledge/`	**Czytana gdy trzeba.** Wiedza i dokumenty dotyczące projektu, dziennik `changelog/` (`.md` + YAML).
Szablon	`templates/`	**Przy tworzeniu dokumentów lub innych artefaktów.** Wzór (np. raportu, notatki, tabeli) gwarantujący powtarzalny format.
Skrypt	`.agents/skills/.../scripts/`	**Uruchamiany przez agenty, nie czytany.** Rzeczy deterministyczne (konwersje, obliczenia) robi kod w `.venv`.
Połączenie MCP	ustawienia MCP narzędzia	**Tylko gdy projekt potrzebuje zewnętrznego programu.** Program wystawia listę narzędzi do dyspozycji agenta. Sekrety tylko w zmiennych środowiskowych. Jak podpiąć: dobre praktyki startera.

Dokąd powinny trafić poszczególne elementy Twoich instrukcji?

1. Musi być widoczne **zawsze** i mieści się w 1–3 liniach? → **zostaje w `AGENTS.md`**
2. Ma obowiązywać w każdej rozmowie, ale jest **długie**? → **reguła „zawsze" + wiersz w routerze**
3. Dotyczy tylko **wybranych zadań** lub typów plików? → **reguła „na zadanie / plik" + wiersz w routerze**
4. To **procedura krok po kroku**? → **skill (`SKILL.md`)**
5. Potrzebuje **osobnego kontekstu** lub lżejszego modelu? → **subagent (`agents/`)**
6. To czynność **deterministyczna** (konwersja PDF/Excel, liczenie)? → **skrypt Pythona (`scripts/`)**
7. To **fakt, słownik pojęć** lub dane o inwestycji? → **baza wiedzy `knowledge/`**
8. Dokument wyjściowy ma mieć **zawsze ten sam układ**? → **szablon w `templates/`**

© 2026 Katarzyna Barcz AutoMATE | All Rights Reserved

6 YAML („dowód osobisty pliku") & Dobre praktyki dla `AGENTS.md`

Nagłówek między `---` to „dowód osobisty" pliku - agent czyta go zanim otworzy treść (oszczędność tokenów). `description` **(zawsze po angielsku!) mówi *kiedy* użyć pliku.** Każda reguła ma **wiersz w routerze** `AGENTS.md` („kiedy → który plik"), który działa w każdym narzędziu. `trigger` (`always_on`, `model_decision`, `glob`) to dodatek czytany tylko przez Antigravity. Pliki `README.md` oznaczaj `audience: "human-only"` i zakaż agentowi ich czytania.

TYP REGUŁY	TYPOWY `DESCRIPTION`	`TRIGGER` (ANTIGRAVITY)
Zawsze	`Always apply <topic> rules.`	`always_on`
Na zadanie	`Use when the task involves <task type>.`	`model_decision`
Na typ pliku	`Use when creating or editing .<filetype> files.`	`glob + globs: "**/*.<ext>"`

`.agents/rules/client-email-style.md`

```
---
trigger: model_decision
description: "Use when the task involves drafting
client emails or project summaries."
---
# Rule: Client email style
- Start with the concrete result...
```

`.agents/skills/bim-audit/SKILL.md`

```
---
name: bim-audit
description: "Validates IFC exports and parameter
naming. Use when the task involves checking IFC
models."
---
# Skill: Audyt modelu krok po kroku
1. Uruchom skrypt walidacji w .venv...
```

Zasady dobrego `AGENTS.md`:

- **Co zawiera:** krótki cel projektu (wejście, wynik, gdzie zatwierdza człowiek), mapę folderów, twarde zakazy globalne (`README*`, zakaz `/tmp`, praca w `.venv`) oraz router do reguł i subagentów.
- **Czego unikać:** procedur, słowników i długich standardów (tworzą szum w kontekście i ryzyko sprzeczności). Bez YAML na początku pliku. Jedna zasada = jedno miejsce.
- **Limity:** Agent wczytuje `AGENTS.md` w całości przy każdym starcie! Dobra praktyka: **poniżej 200 linii**. Codex ucina po **32 KiB**, Antigravity: **24 KB** na plik.

`AGENTS.md (fragment routera)`

```
## 4. Rules
| When | File to read |
| :--- | :--- |
| Always, at the start of every session | .agents/rules/file-versioning.md |
| When the task involves drafting client emails | .agents/rules/client-email-style.md |
```

7 Higiena Kontekstu i Tokenów (DO's & DON'Ts, `.venv`, ChangeLog)

DO'S

- **Jedna zasada, jeden plik, jedno miejsce** - krótki `AGENTS.md` (mapa, zakazy, router), reszta w regułach i skillach.
- **Zwięzły output i dobór modeli:** wymuszaj krótkie odpowiedzi (tokeny wyjściowe są najdroższe!); subagent domyślnie dziedziczy model, lżejszy ustawiaj świadomie do prostych zadań.
- **Determinizm w skryptach:** obliczenia i konwersję plików rób kodem w `.venv`, nie ręcznie w oknie modelu.
- **Małe kroki i ekonomia:** jedna zmiana → test na żywym zadaniu; pilnuj `Cost of Tokens < Worth of the Outcome`.

DON'TS

- Nie wrzucaj wszystkiego do `AGENTS.md` ani nie ustawiaj każdej reguły jako „zawsze" (szum = halucynacje).
- Nie dubluj tych samych zasad w kilku plikach (ryzyko sprzeczności).
- Nie karm modelu surowym PDF, Wordem ani Excelem i nie chowaj jednorazowych skryptów w `/tmp`.
- Nie podmieniaj starego pliku, zanim nowa wersja (`_vXX.YY`) przejdzie testy.

Środowisko wirtualne `.venv` zamiast `/tmp` i tymczasowych skryptów:

- Bez konkretnych zasad agent napisze jednorazowy skrypt w `/tmp`, który zniknie po restarcie - przy następnej rozmowie znów zapłacisz tokenami za napisanie go od zera.
- Kod należy napisać **raz**, zapisać w `.agents/skills/.../scripts/`, a lekki subagent odpala go przez `.venv/bin/python`. Zanim agent napisze nowy skrypt, sprawdza, czy taki już jest.

ChangeLog (`knowledge/changelog/`):

1. **Dziennik prowadzony w tle:**
   Agent po każdym zakończonym kroku dopisuje w punktach: co zrobione, co dalej, jakie decyzje zapadły. Jeden plik na dzień (`YYYY-MM-DD_ChangeLog.md`).
2. **Ten sam folder na dysku:**
   Otwierasz ten sam katalog w drugim narzędziu lub nowej sesji (np. gdy skończą się tokeny w jednej licencji).
3. **Te same zasady:**
   Kolejny agent czyta `AGENTS.md`, reguły i skille oraz **tylko ostatni wpis** ChangeLogu.
4. **„Kontynuuj pracę":**
   Praca toczy się dalej od miejsca, w którym stanęła.
   Ciągłość danych i pamięć projektu trzymają pliki, nie czat.

© 2026 Katarzyna Barcz AutoMATE | All Rights Reserved